

MINISTRY OF EDUCATION AND TRAINING
UNIVERSITY OF ECONOMICS AND FINANCE
HO CHI MINH CITY



FINAL PROJECT REPORT
SOFTWARE QUALITY VERIFICATION
TOPIC: ONLINE SALES MANAGEMENT SYSTEM

Class: 252.ITE1231E.A02E

Instructor: **MSc. Nguyễn Ngọc Tú**

Performed by Student:

Hoàng Văn Thiên	22D1ITE-SWE03	225051915
Nguyễn Thành Đạt	22D1ITE-SWE03	225050896
Lê Quang Duy	22D1ITE-SWE03	225051169

Ho Chi Minh City, April 2026

RECORD OF CHANGES

This section summarizes the major revision milestones of the final report and the submission package. Only meaningful changes that affected scope, execution, evidence, defects, metrics, or final packaging are recorded here.

Table R.1: Record of changes

Version	Date	Author	Change Summary
0.1	2026-04-05	QA Team	Established the report structure, completed the initial project audit, and defined the first testing scope and plan baseline.
0.5	2026-04-05	QA Team	Added the UI and API test cases, prepared the test data set, and documented the initial automation approach.
0.8	2026-04-06	QA Team	Added execution evidence, initial defect records, early metrics, and supporting automation outputs.
1.0	2026-04-06	Hoang Van Thien	Consolidated the main report content, appendix linkage, and final submission package structure.
1.1	2026-04-10	QA Team	Expanded verified UI execution coverage and synchronized the report with the updated results, metrics, and evidence set.
1.2	2026-04-11	QA Team	Removed the remaining Not Run status, synchronized the live GitHub Issues, and refreshed the defect and metrics sections.
1.3	2026-04-12	QA Team	Finalized chapter wording, references, appendix content, formatting consistency, and submission-ready packaging.

LIST OF CONTENTS

RECORD OF CHANGES	2
LIST OF CONTENTS	i
LIST OF FIGURES	iii
LIST OF TABLES	iv
CHAPTER 1. OVERVIEW	1
1.1. Project Information	1
1.1.1. System overview	1
1.1.2. Main business modules	3
1.1.3. API overview.....	4
1.2. Project Team and Task Allocation	6
1.2.1. Team members	6
1.2.2. Responsibility matrix	7
CHAPTER 2. TEST PLAN.....	9
2.1. Test Scope	9
2.1.1. In-Scope	9
2.1.2. Out-of-Scope	10
2.2. Test Strategy & Approach.....	11
2.2.1. Manual testing approach	11
2.2.2. Basic automation approach	12
2.2.3. Black-box testing.....	13
2.2.4. White-box testing	14
2.2.5. Entry and exit criteria.....	15
2.3. Test Environment	16
2.3.1. Hardware and software environment	16
2.3.2. Browsers, OS, devices.....	17
2.3.3. Test accounts and test data	18
2.3.4. Tools used	19
CHAPTER 3. TEST DESIGN & EXECUTION	21
3.1. Test Scenarios.....	21
3.1.1. UI test scenarios	21
3.1.2. API test scenarios	22

3.1.3. Edge cases and exception scenarios	23
3.2. Test Case Specification	25
3.2.1. UI test cases.....	25
3.2.2. API test cases	28
3.2.3. Test case allocation by member	29
3.2.4. Execution evidence summary	30
CHAPTER 4. DEFECT REPORT & METRICS.....	32
4.1. Bug Management Tool and Workflow.....	32
4.1.1. Selected tool	32
4.1.2. Issue workflow	33
4.1.3. Severity and priority rules	34
4.2. Defect Log.....	35
4.2.1. Critical and high severity defects	35
4.2.2. Medium and low severity defects	37
4.2.3. Screenshots and reproduction evidence	38
4.3. Test Summary Metrics	40
4.3.1. Total test cases executed	40
4.3.2. Pass/Fail/Blocked statistics	41
4.3.3. Defect count by severity.....	41
4.3.4. Defect ratio and observations	41
CHAPTER 5. CONCLUSION AND FUTURE WORK	43
5.1. Achievements and compliance with requirements.....	43
5.2. Challenges and limitations	44
5.3. Suggestions for future enhancements.....	46
APPENDIX	49
REFERENCES	53
TASK DIVISION AND CONTRIBUTION SUMMARY	54

LIST OF FIGURES

Figure 1.1-1: Representative administrative interface evidence after successful login to the system.....	2
Figure 1.1-2: Representative administrative modules and management screens included in the test scope.	4
Figure 1.1-3: Newman execution evidence for the API test suite.....	5
Figure 2.2-1: Representative API automation execution output using Newman.....	13
Figure 2.3-1: Representative secured administrative environment and module access used during UI testing.....	18
Figure 2.3-2: Representative controlled validation dataset behavior during product import testing.	19
Figure 3.1-1: Example UI execution evidence for a successful authentication scenario.....	22
Figure 3.1-2: API scenario execution evidence produced by the Newman full-run output.	23
Figure 3.1-3: Example edge-case evidence from the product import preview validation flow.	25
Figure 3.2-1: Defect-focused UI execution evidence for invoice creation failure.....	28
Figure 4.1-1: GitHub Issue example used as the official bug-management record for a confirmed defect.....	33
Figure 4.2-1: Reproduction evidence for the high-severity invoice creation defect.....	36
Figure 4.2-2: GitHub Issue evidence for a high-severity invoice cancellation defect.....	37
Figure 4.2-3: GitHub Issue evidence for the medium-severity purchase validation defect....	38
Figure 4.2-4: GitHub Issue evidence for the product-import confirmed defect.	40
Figure 4.3-1: Metrics summary exported from the synchronized results workbook	42
Figure 5.1-1: Final execution and metrics summary used to support the conclusion of the testing project.	44
Figure 5.2-1: Example of a confirmed open defect that remains unresolved at the final reporting point.	46
Figure 5.3-1: Existing automation evidence that can be expanded in future iterations to increase regression strength.	47

LIST OF TABLES

Table 1.1-1: System overview.....	2
Table 1.1-2: Main business modules.....	3
Table 1.1-3: API overview	5
Table 1.2-1: Team members.....	7
Table 1.2-2: Responsibility matrix.....	8
Table 2.1-1: In-scope test areas.....	10
Table 2.1-2: Out-of-scope items.....	11
Table 2.2-1: Automation strategy overview.....	12
Table 2.2-2: Black-box and white-box usage	14
Table 2.2-3: Entry and exit criteria	15
Table 2.3-1: Hardware and software environment.....	16
Table 2.3-2: Browsers, OS, and devices	17
Table 2.3-3: Test accounts and test data	18
Table 2.3-4: Tools used.....	20
Table 3.1-1: UI scenario distribution by interface	21
Table 3.1-2: Representative UI scenario groups.....	22
Table 3.1-3: API scenario coverage	23
Table 3.1-4: Representative edge and exception scenarios.....	24
Table 3.2-1: Sample UI test case specification (positive case).....	25
Table 3.2-2: Sample UI test case specification (defect-focused case).....	26
Table 3.2-3: Sample API test case specification.....	28
Table 3.2-4: Test case allocation by member.....	29
Table 3.2-5: Execution evidence summary	30
Table 3.2-6: Execution evidence types	30
Table 4.1-1: Bug-management tool selection	32
Table 4.1-2: Issue workflow.....	34
Table 4.1-3: Severity and priority rules	34
Table 4.2-1: High-severity confirmed defects	35
Table 4.2-2: Medium and low severity confirmed defects	37
Table 4.2-3: Defect evidence mapping summary	39
Table 4.3-1: Execution totals	40

Table 4.3-2: Pass / Fail / Blocked statistics.....	41
Table 4.3-3: Confirmed defect count by severity.....	41
Table 4.3-4: Defect ratio and observations	42
Table 5.1-1: Final requirement compliance summary	43
Table 5.1-2: Final delivery baseline	44
Table 5.2-1: Current limitations and impact	45
Table 5.3-1: Recommended future enhancement roadmap.....	47
Table A.1: GitHub submission links	49
Table A.2: Excel and scenario artifacts.....	49
Table A.3: Test scripts and test data	50
Table A.4: Final results and evidence package	51
Table A.5: Automation video artifacts	52
Table A.6: Bug tracker and defect artifacts.....	52
Table T.1: Task division and contribution summary	54

CHAPTER 1. OVERVIEW

1.1. Project Information

This report presents the software-testing study and final submission for the Online Sales Management System, a web-based business application developed on ASP.NET Core MVC (.NET 8). The system is designed to support core sales and inventory operations, including user authentication, product management, supplier management, customer management, purchase handling, invoice processing, stock monitoring, reporting, and public product browsing.

The main purpose of this project is to evaluate whether the system behaves correctly in its business-critical workflows and whether the final submission satisfies the course requirements for test planning, test-case design, execution evidence, bug management, reporting quality, and basic automation. The testing effort was conducted using a combination of manual testing, API testing, and targeted automation. All major claims in this report are supported by execution artifacts such as screenshots, Excel workbooks, automation outputs, runner files, GitHub Issues, and a recorded automation video.

This project is well suited for a software-testing study because it combines several functional surfaces with business rules that can fail in meaningful ways. Purchase and invoice workflows affect inventory and transaction states, role-based access control determines which functions each user can reach, and product import adds validation and batch-processing risk. These characteristics made the system suitable for structured test design, real execution, defect reporting, and evidence-based analysis.

1.1.1. *System overview*

The Online Sales Management System is an internal-and-public web application that supports both operational users and public visitors. The internal area provides a secured administrative interface where authorized users can manage products, suppliers, customers, purchases, invoices, stock records, and business reports. The public area provides product discovery features for browsing, filtering, sorting, and viewing product details.

The system follows a layered ASP.NET Core MVC architecture and uses a relational database for persistent storage. Authentication and authorization are handled through ASP.NET Identity, while Entity Framework Core supports the data-access layer. In addition

to the browser-based interface, the project exposes a small API surface for health checking and public catalog retrieval. This gave the team a practical test scope: broad enough to include both UI and API verification, but still manageable within the project timeline.

For the purpose of the final submission, the system was tested as a locally executed environment using seeded demo data. This allowed the team to verify realistic workflows such as valid login, invalid login, role restrictions, product creation, purchase draft creation, invoice processing, stock-related views, reporting filters, product import validation, and catalog API behavior. The system therefore represents a practical case study for end-to-end software testing rather than a purely theoretical exercise.

Table 1.1-1: System overview

Item	System description
System name	Online Sales Management System
System type	Web-based sales and inventory management system
Architecture style	ASP.NET Core MVC application with database-backed business workflows
Primary users	Administrative users and public catalog visitors
Main interfaces	Admin UI, Public UI, Health API, Catalog API
Testing objective	Verify business-critical workflows, validation behavior, permissions, and API responses

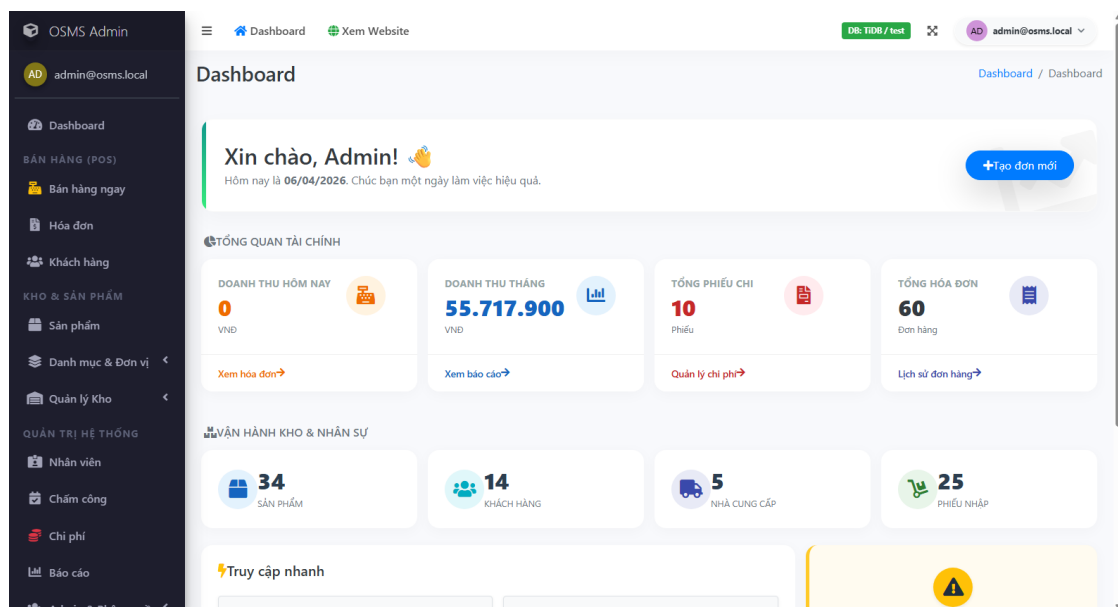


Figure 1.1-1: Representative administrative interface evidence after successful login to the system.

1.1.2. Main business modules

The system contains several business modules that are directly related to daily operational activities. These modules were selected as the primary testing targets because incorrect behavior in any of them could affect business continuity, inventory accuracy, transaction correctness, or access control.

The Authentication and Permissions modules control whether a user can log in successfully and whether the user can access only the functions allowed by his or her assigned role. These modules are foundational because every secured business workflow depends on them. The Products, Suppliers, and Customers modules provide core master-data management and therefore affect downstream processes such as purchases and invoices.

The Purchases and Invoices modules are the most business-critical workflows in the system. Purchase processing affects inbound stock and supplier-side records, while invoice processing affects outbound stock, payment states, and customer transactions. The Product Import module introduces additional risk because it handles multi-row validation and batch-style updates. The Stock and Reports modules are also high value because they influence operational decisions and managerial visibility. Finally, the Public Catalog module, although smaller in scope, is important because it represents the customer-facing browsing surface of the system.

Table 1.1-2: Main business modules

Module	Business purpose	Testing importance
Authentication	Allows valid users to access the system securely	High
Permissions / Authorization	Restricts access based on user role	High
Products	Manages product master data and sales items	High
Suppliers	Manages supplier information used by purchases	Medium
Customers	Manages customer records used by invoices	Medium
Purchases	Handles inbound stock and purchasing workflows	High
Invoices	Handles sales transactions, payment, and stock deduction	High

Module	Business purpose	Testing importance
Product Import	Supports bulk product validation and import processing	High
Stock	Shows inventory state and stock movement information	High
Reports	Provides summary and filtered business reports	Medium
Public Catalog	Supports public browsing, filtering, and product details	Medium

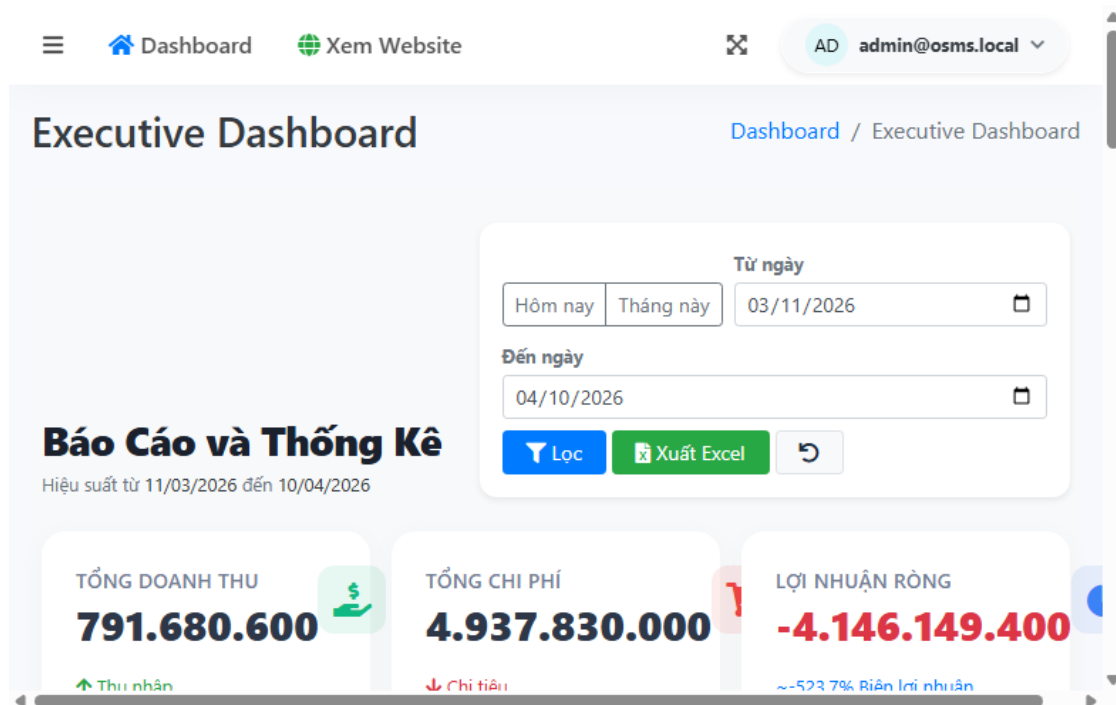


Figure 1.1-2: Representative administrative modules and management screens included in the test scope.

1.1.3. API overview

Although the project is primarily UI-driven, it also exposes a limited but meaningful API surface that supports monitoring and public catalog retrieval. This API layer was included in the final submission because it provides stable request-response targets that are suitable for systematic verification with Postman and Newman.

The Health API is used to confirm service availability and payload consistency. It allows the testing team to verify whether the application is running correctly and whether the returned

metadata remains structurally valid. The Catalog API provides public-facing product retrieval functions, including product listing, product-detail retrieval, trending data, and filter information. These endpoints were valuable for validating status codes, response payloads, query handling, and negative scenarios such as invalid parameters or unsupported sort values.

Testing the API surface strengthens the report in two ways. First, it complements the UI coverage by validating backend-accessible behavior directly. Second, it provides repeatable execution outputs through Newman, including text summaries and XML runner artifacts. This makes the API section easier to verify and easier to defend during review.

Table 1.1-3: API overview

API area	Purpose	Representative endpoints
Health API	Service status verification	GET /api/v1/health
Catalog product list	Public catalog retrieval	GET /api/v1/catalog/products
Catalog product detail	Single product information retrieval	GET /api/v1/catalog/products/{id}
Trending products	Returns popular or highlighted items	GET /api/v1/catalog/trending
Catalog filters	Provides available filtering metadata	GET /api/v1/catalog/filters

```

Newman Full Run Summary
newman

OSMS API Automation

□ 01 Health Smoke
  ↳ TC-API-HLT-001 Health endpoint returns service availability metadata
    GET http://localhost:5068/api/v1/health [200 OK, 318B, 38ms]
    ✓ Status code is 200
    ✓ Health payload contains expected fields

□ 02 Catalog Happy Path
  ↳ TC-API-CAT-001 Catalog products endpoint returns paginated active products with default parameters
    GET http://localhost:5068/api/v1/catalog/products?page=1&pageSize=2 [200 OK, 731B, 318ms]
    ✓ Status code is 200
    ✓ Paginated payload is returned

  ↳ TC-API-CAT-002 Catalog products endpoint supports search by SKU or product name
    GET http://localhost:5068/api/v1/catalog/products?search=SP001&page=1&pageSize=5 [200 OK, 494B, 265ms]
    ✓ Status code is 200
    ✓ Search returns matching products only

  ↳ TC-API-CAT-003 Catalog products endpoint filters by valid categoryId
    GET http://localhost:5068/api/v1/catalog/products?categoryId=30012&page=1&pageSize=5 [200 OK, 949B, 257ms]
    ✓ Status code is 200
    ✓ Every returned item matches the requested category when items are present

  ↳ TC-API-CAT-004 Catalog products endpoint filters by valid brandId
    GET http://localhost:5068/api/v1/catalog/products?brandId=30039&page=1&pageSize=5 [200 OK, 967B, 256ms]

```

Figure 1.1-3: Newman execution evidence for the API test suite.

1.2. Project Team and Task Allocation

The testing project was completed by a three-member QA team. Task allocation was planned carefully to satisfy the course requirement that each member contribute at least ten non-overlapping test cases. Instead of distributing work randomly, the team assigned module ownership so that each member could focus on a specific functional area while still contributing to the integrated final submission.

This structure made ownership clearer and helped the team keep the final artifacts consistent. Each member was responsible for designing, executing, and documenting test cases within defined module boundaries. Shared artifacts such as the final report, metrics workbook, defect register, and automation evidence were synchronized only after the execution baseline had been verified. This reduced overlap and made it easier to connect each result, screenshot, and defect record back to a clear owner.

In addition to module ownership, one member also coordinated final integration and submission consistency checks. This role was necessary because the final package includes multiple artifact types and requires strong cross-file traceability between scenarios, test cases, results, metrics, defect records, automation outputs, and presentation materials.

1.2.1. Team members

The project team consists of three members from the same class group. Each member was assigned a distinct module cluster and was required to contribute a sufficient number of individual test cases. The final workload distribution satisfies the requirement that every member owns more than ten UI test cases.

Hoang Van Thien handled the largest share of UI ownership and also led the project audit, artifact alignment, and final report consolidation. Nguyen Thanh Dat focused mainly on customer, supplier, purchase, and part of the stock workflows. Le Quang Duy focused mainly on product, product import, stock-related, and public-catalog workflows. This division kept responsibilities clear while helping the team maintain a consistent final package.

Table 1.2-1: Team members

Member	Class	Student ID	Main ownership	UI cases owned
Hoang Van Thien	22D1ITE-SWE03	225051915	Authentication, Permissions, Admin Users, Admin Groups, Invoices, Reports, partial Public Catalog	17
Nguyen Thanh Dat	22D1ITE-SWE03	225050896	Customers, Suppliers, Purchases, partial Stock	13
Le Quang Duy	22D1ITE-SWE03	225051169	Products, Product Import, partial Stock, Public Catalog	14

1.2.2. Responsibility matrix

To ensure non-overlapping ownership and a clear final submission structure, the team used a module-based responsibility matrix. Each member was primarily responsible for the test design and execution of a specific functional area, while final integration activities were coordinated centrally to keep the report, metrics, defects, and supporting evidence consistent.

This matrix also reflects the practical reality of the project. Some deliverables, such as the final report, metrics workbook, defect synchronization, and automation packaging, required cross-member review. However, the ownership model remained clear enough that each module can be traced back to a primary contributor. This is important not only for grading compliance but also for presentation defense, where each member must be able to explain the rationale, execution, and evidence behind the parts they owned.

Table 1.2-2: Responsibility matrix

Work item	Primary owner	Supporting members
Project audit and scope definition	Hoang Van Thien	Nguyen Thanh Dat, Le Quang Duy
Authentication and permission testing	Hoang Van Thien	-
Admin users and admin groups testing	Hoang Van Thien	-
Invoice and report testing	Hoang Van Thien	-
Customer and supplier testing	Nguyen Thanh Dat	-
Purchase workflow testing	Nguyen Thanh Dat	-
Stock-related testing	Nguyen Thanh Dat / Le Quang Duy	Shared coverage by module split
Product testing	Le Quang Duy	-
Product import testing	Le Quang Duy	-
Public catalog testing	Le Quang Duy / Hoang Van Thien	Shared coverage by scenario split
API testing and Newman execution	QA team	Shared execution and verification
Automation framework maintenance	QA team	Coordinated integration
Final results, metrics, and defect synchronization	Hoang Van Thien	QA team
Final report and presentation consolidation	Hoang Van Thien	QA team

Overall, the team allocation satisfies the course requirements for minimum contribution, non-overlapping test ownership, and traceable responsibility. This structure also supports stronger evidence management because each module, test case set, execution artifact, and defect record can be linked back to a clearly assigned owner and a verifiable workflow.

CHAPTER 2. TEST PLAN

2.1. Test Scope

This chapter defines the boundaries of the testing effort for the Online Sales Management System. The scope was determined based on business impact, functional importance, visible user workflows, and source-informed risk analysis. Priority was given to areas where failures could directly affect access control, transaction correctness, inventory consistency, bulk data handling, and managerial visibility.

The final scope includes both user-facing and backend-accessible components. UI testing was prioritized for the administrative interface and the public product catalog, while API testing was used for the exposed health and catalog endpoints. This combination helped the team cover both visible business behavior and direct request-response validation without expanding the project beyond a manageable level.

2.1.1. *In-Scope*

- The following items were included in the official testing scope of this submission:
- Authentication and login validation
- Role-based access control and permission boundaries
- Admin user and admin group management
- Customer and supplier master-data workflows
- Product creation, update validation, duplicate checking, and product-state management
- Product import validation, preview behavior, and import confirmation flow
- Purchase creation, receiving, cancellation, and related validation behavior
- Invoice creation, payment, cancellation, and inventory-related business behavior
- Stock visibility, low-stock views, and stock-movement filtering
- Report filtering and export-related source verification
- Public product catalog search, filtering, sorting, and detail pages
- API verification for health and public catalog endpoints
- Execution evidence collection, defect logging, metrics generation, and traceability mapping

Table 2.1-1: In-scope test areas

Scope item	Description
Admin UI	Authentication, permissions, master data, purchases, invoices, stock, reports
Public UI	Product search, filtering, sorting, and detail pages
API surface	Health endpoint and public catalog endpoints
Business validation	Input validation, duplicate handling, role restrictions, inventory-sensitive rules
Defect workflow	Execution evidence, GitHub Issues, defect log, metrics, and result traceability
Automation support	UI and API automation for selected high-value flows

2.1.2. Out-of-Scope

The following items were excluded from the scope of this submission due to time, environment, or project-boundary limitations:

- Performance and load testing
- Penetration testing and deep security assessment
- Mobile application testing
- Full responsive certification across multiple mobile and tablet devices
- Accessibility compliance audit
- Disaster recovery, backup, and failover testing
- Browser compatibility beyond the verified baseline and basic smoke validation
- Undocumented write apis outside the exposed catalog and health endpoints
- Production deployment validation and cloud-infrastructure testing

These exclusions do not reduce the validity of the current testing scope. Instead, they clarify that the submission is focused on functional correctness, execution evidence, and business-rule verification rather than non-functional certification.

Table 2.1-2: Out-of-scope items

Excluded item	Reason for exclusion
Performance / load testing	Not required by the course scope and no dedicated load environment was provided
Deep security testing	Requires specialized tools and a different testing objective
Mobile / device matrix	Not part of the required deliverables for this final submission
Full cross-browser certification	Only basic browser verification was feasible within the available environment
Infrastructure / deployment testing	Project was executed in a local testing environment, not a production-like deployment target

2.2. Test Strategy & Approach

The testing strategy was designed to keep coverage broad while still producing evidence that the team could explain and defend. Because the system includes both interactive business workflows and a small API surface, the team combined manual testing, API testing, and selective automation. This allowed the report to cover visible user behavior, backend-accessible responses, and repeatable high-value flows without overstating the maturity of the automation layer.

The strategy was primarily risk-based. Business-critical workflows such as authentication, permission boundaries, purchases, invoices, stock-sensitive operations, and product import were prioritized because defects in these areas could lead to data inconsistency, incorrect transaction behavior, or operational failures. Lower-risk areas such as cosmetic formatting and deep non-functional properties were not prioritized in the same way.

2.2.1. Manual testing approach

Manual testing was the primary execution method for the submission because it provides the broadest coverage for visible business behavior, validation feedback, access-control behavior, workflow branching, and exception handling. It was especially important for

modules where human observation is necessary to verify form behavior, redirects, inline validation, data persistence, and functional correctness across multiple screens.

The manual approach followed a structured sequence:

- Prepare the environment and seeded test data
- Identify the target test case and required preconditions
- Execute the defined steps exactly as documented
- Compare actual system behavior against the expected result
- Capture screenshots or output artifacts
- Record the final result in the workbook
- Log a defect when the mismatch is reproducible and supported by evidence

2.2.2. Basic automation approach

Basic automation was implemented to strengthen repeatability and bonus coverage rather than to replace the full manual test effort. Two automation layers were used in this project.

First, UI automation was implemented using Selenium WebDriver with .NET 8 and xUnit. This layer was used for selected high-value workflows such as login verification, permission checks, purchase creation, invoice-related validation, import preview behavior, and focused reruns for regression confirmation.

Second, API automation was implemented using Postman collections executed through Newman. This layer was used to verify health and catalog endpoints in a repeatable way and to produce exportable evidence such as text summaries and XML result files.

The automation scope was kept selective so that the automated outputs remained stable, reviewable, and clearly linked to the official test cases. The goal was not to claim full regression automation, but to provide credible support for repeated UI and API verification with real artifacts such as screenshots, TRX files, Newman text output, and XML runner results.

Table 2.2-1: Automation strategy overview

Automation type	Technology	Main purpose
UI automation	Selenium WebDriver + .NET 8 + xUnit	Repeatable verification of selected high-value UI workflows

Automation type	Technology	Main purpose
API automation	Postman + Newman	Repeatable verification of exposed API endpoints and exportable runner evidence
Evidence output	Screenshots, TRX, TXT, XML	Support result comparison, execution traceability, and bonus coverage

```

Newman Full Run Summary
newman

OSMS API Automation

□ 01 Health Smoke
  ↳ TC-API-HLT-001 Health endpoint returns service availability metadata
    GET http://localhost:5068/api/v1/health [200 OK, 318B, 38ms]
    ✓ Status code is 200
    ✓ Health payload contains expected fields

□ 02 Catalog Happy Path
  ↳ TC-API-CAT-001 Catalog products endpoint returns paginated active products with default parameters
    GET http://localhost:5068/api/v1/catalog/products?page=1&pageSize=2 [200 OK, 731B, 318ms]
    ✓ Status code is 200
    ✓ Paginated payload is returned

  ↳ TC-API-CAT-002 Catalog products endpoint supports search by SKU or product name
    GET http://localhost:5068/api/v1/catalog/products?search=SP001&page=1&pageSize=5 [200 OK, 494B, 265ms]
    ✓ Status code is 200
    ✓ Search returns matching products only

  ↳ TC-API-CAT-003 Catalog products endpoint filters by valid categoryId
    GET http://localhost:5068/api/v1/catalog/products?categoryId=30012&page=1&pageSize=5 [200 OK, 949B, 257ms]
    ✓ Status code is 200
    ✓ Every returned item matches the requested category when items are present

  ↳ TC-API-CAT-004 Catalog products endpoint filters by valid brandId
    GET http://localhost:5068/api/v1/catalog/products?brandId=30039&page=1&pageSize=5 [200 OK, 967B, 256ms]

```

Figure 2.2-1: Representative API automation execution output using Newman.

2.2.3. Black-box testing

Black-box testing was the dominant testing strategy used in this project. Test execution focused on system inputs, user actions, visible outputs, and business outcomes without depending on internal implementation details during runtime verification.

This strategy was applied to:

- Login success and login rejection
- Role-based access restriction
- Crud validation for products, suppliers, and customers
- Purchase and invoice workflow outcomes
- Product import validation and confirmation behavior

- Public catalog search, sorting, and filtering behavior
- Api status codes and payload validation

Black-box testing was the most suitable execution approach for this project because the course emphasizes practical test design, visible system behavior, and real execution evidence. It also matched the way the team verified business workflows through forms, redirects, validation messages, permissions, and API responses.

2.2.4. White-box testing

White-box testing was not used as a standalone implementation-heavy testing stream such as unit-test-driven structural coverage. However, white-box-informed analysis was used to support test design. The team inspected the source code, routes, controllers, validation logic, and seeded data in order to identify hidden edge cases, role conditions, unsupported inputs, and risky business-rule paths before executing the test cases.

This code-informed analysis helped the team design stronger scenarios for:

- Permission boundaries across admin, sales, and warehouse roles
- Invoice creation and cancellation behavior
- Import validation rules and preview-confirm flow
- Unsupported api query values
- Route access and redirection logic
- Stock-sensitive workflows and business-state transitions

Overall, the project used a black-box execution model supported by source-informed test design. This was a practical way to combine real user-facing execution with code inspection, especially for permission rules, validation paths, and business-state transitions that are not always obvious from the interface alone.

Table 2.2-2: Black-box and white-box usage

Approach	How it was used in this project
Black-box	Main execution strategy for UI and API verification based on inputs, actions, and visible outputs
White-box-informed	Used during test design through source inspection to identify edge cases, hidden branches, and risky logic

Approach	How it was used in this project
Reason for combination	Improves design quality without overstating the presence of deep structural automation or unit-level coverage

2.2.5. Entry and exit criteria

Entry and exit criteria were defined to keep execution disciplined and to ensure that reported results were supported by reproducible evidence.

Entry criteria for execution:

- The application builds and runs successfully in the local environment
- The target base url is reachable
- Seeded accounts and required test data are available
- The required test case, preconditions, and test data are prepared
- The evidence folders and result workbooks are ready for recording outputs
- The target module can be opened in the environment being used

Exit criteria for the testing cycle:

- All designed UI and API test cases in the documented scope have been executed.
- Actual results and final statuses have been recorded
- Screenshots, runner outputs, and evidence files have been stored
- Confirmed mismatches have been logged into the defect register and github Issues
- Final results, metrics, and traceability files have been synchronized
- The report and presentation reflect the latest execution baseline

The execution cycle satisfied the defined exit criteria for coverage and documentation. However, product-quality closure was not fully achieved because four confirmed defects remained open at the final reporting point.

Table 2.2-3: Entry and exit criteria

Criteria type	Definition
Entry criteria	Application is runnable, test data is available, target module is reachable, and evidence recording is ready
Exit criteria	All test cases are executed, results are recorded, defects are logged, and artifacts are synchronized

Criteria type	Definition
Current status	Execution exit criteria satisfied; product quality still limited by unresolved confirmed defects

2.3. Test Environment

A controlled local test environment was used for all verified execution in this submission. The environment was configured to support both manual and automated testing, with the same application instance serving as the source for UI and API verification. Using a single consistent environment reduced result inconsistency and strengthened traceability across screenshots, runner outputs, workbooks, and defect records.

The environment section is important because it defines the reproducibility conditions of the submission. All reported outcomes in the final workbooks and screenshots were produced from this testing baseline unless explicitly stated otherwise.

2.3.1. Hardware and software environment

The execution environment used for the project was a Windows-based local workstation capable of running the ASP.NET Core MVC application, browser-based UI testing, Newman API execution, and supporting evidence tools. The application was executed locally using the .NET 8 runtime and connected to the database environment identified in the system as TiDB / test.

This environment was sufficient for the scope of the final submission because it supported functional execution, screenshot capture, UI automation, API execution, and evidence export. No distributed infrastructure or staging server was required for the tested endpoints and business flows included in the report.

Table 2.3-1: Hardware and software environment

Environment item	Value
Operating system	Windows 11
Application framework	ASP.NET Core MVC (.NET 8)
ORM / data access	EF Core
Authentication framework	ASP.NET Identity

Environment item	Value
Database tag observed in execution	TiDB / test
Execution type	Local environment
Base URL	http://localhost:5068
Repository path	E:\Project\Online-Sales-Management-System

2.3.2. Browsers, OS, devices

The primary browser used for UI execution was Google Chrome. Chrome was used for the main body of manual execution and for the majority of Selenium-based automated runs. In addition, a basic smoke rerun was executed on Microsoft Edge to provide minimal cross-browser evidence for the final submission.

Because the project is a web system and the course does not require a full device-compatibility matrix, testing was performed under desktop conditions only. No mobile-device certification or tablet matrix was included. This browser setup was sufficient for the final submission because Chrome covered the main execution baseline and Microsoft Edge provided a basic compatibility check through a smoke rerun.

Table 2.3-2: Browsers, OS, and devices

Item	Usage in testing
Windows 11	Main execution operating system
Google Chrome	Primary browser for manual UI and Selenium-based execution
Microsoft Edge	Basic smoke verification for limited cross-browser evidence
Desktop / laptop environment	Primary device class used for the final submission
Mobile devices	Not included in the official scope
Tablet devices	Not included in the official scope

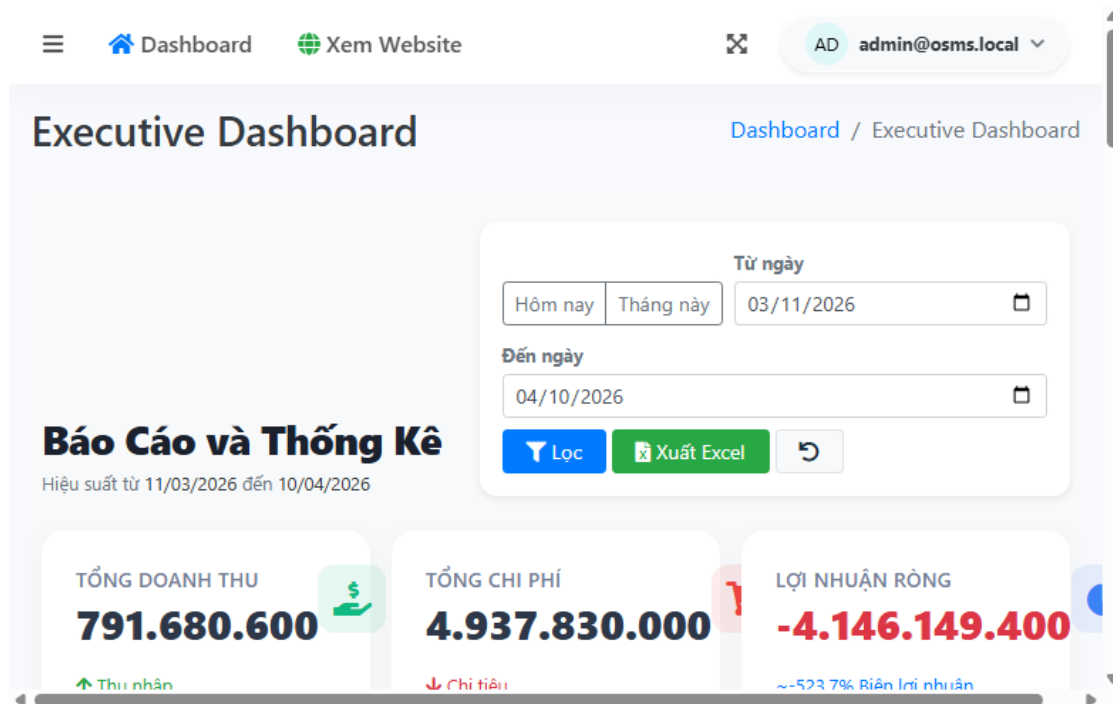


Figure 2.3-1: Representative secured administrative environment and module access used during UI testing.

2.3.3. Test accounts and test data

The submission relied on seeded demo accounts and controlled test data rather than live production accounts. This approach reduced risk and made execution reproducible. The seeded accounts supported role-based verification across administrative, sales, and warehouse perspectives.

The test data set also included prepared values for product creation, purchase and invoice scenarios, API query combinations, and product import validation files. These datasets allowed the team to verify both happy-path and negative-path behavior in a controlled way.

Sensitive real-world credentials were not stored in the report. Only test-safe and seeded execution data was used for the final submission.

Table 2.3-3: Test accounts and test data

Data category	Example / source	Purpose
Admin account	admin@osms.local	Authentication, admin access, invoice, report, and configuration-related workflows

Data category	Example / source	Purpose
Sales account	sales@osms.local	Permission verification and sales-side behavior
Warehouse account	warehouse@osms.local	Purchase and stock-related verification
Negative login data	Invalid password, unknown email, inactive-account scenarios	Authentication validation
Import dataset	Mixed-validation workbook and invalid-format sample	Product import preview and confirmation testing
API query data	Real category IDs, brand IDs, product IDs, and invalid parameters	Catalog and health API verification
Seeded business data	Products, suppliers, customers, purchases, invoices, stock movement	Business-flow execution support

Xem trước import Excel

Xem trước dữ liệu import
Chỉ những dòng **hợp lệ** mới được import. Dòng lỗi sẽ hiển thị lý do.

Tổng dòng: 6 Hợp lệ: 1 Lỗi: 5

#	SKU	Tên	Danh mục	Đơn vị	Brand	Giá nhập	Giá bán	Tồn kho	Active	Trending	Trạng thái
2	QAIMP001	QA Import Product 01	30012 Laptop	30015 Cái	30039 Acer	100.000	120.000	10	✓	✗	Hợp lệ
3	QAIMP002		30012 Laptop	30015 Cái	30039 Acer	100.000	120.000	10	✓	✗	Lỗi • Name bắt buộc.
4	QAIMP003	QA Import Product Invalid Category	- UnknownCategory	30015 Cái	30039 Acer	100.000	120.000	10	✓	✗	Lỗi • CategoryName không tồn tại: UnknownCategory
5	QAIMP004	QA Import Product Negative Cost	30012 Laptop	30015 Cái	30039 Acer	-1.000	120.000	10	✓	✗	Lỗi • CostPrice không được âm.
6	QAIMP005	QA Import Product Duplicate 01	30012 Laptop	30015 Cái	30039 Acer	100.000	120.000	10	✓	✗	Lỗi • SKU bị trùng trong file Excel.
7	QAIMP006	QA Import Product Duplicate 02	30012 Laptop	30015 Cái	30039 Acer	100.000	120.000	10	✓	✗	Lỗi • SKU bị trùng trong file Excel.

Hủy | Import 1 dòng hợp lệ

Figure 2.3-2: Representative controlled validation dataset behavior during product import testing.

2.3.4. Tools used

The project used a practical set of tools chosen to match the requirements of the course and the structure of the system under test. No unnecessary tooling was introduced. Each tool supported a specific stage of the workflow, including planning, execution, automation, evidence collection, defect logging, and final reporting.

Excel-compatible files were used for traceable test case, result, and metrics management. Selenium WebDriver with xUnit was used for basic UI automation. Postman and Newman were used for API execution and exportable runner outputs. GitHub Issues was used as the real bug-management tool required by the course. Screenshots, TRX files, XML outputs, TXT logs, and the recorded automation video were used to support evidence completeness.

Table 2.3-4: Tools used

Tool / technology	Purpose
Microsoft Word	Final report authoring and formatting
Microsoft PowerPoint	Final presentation preparation
Excel-compatible XLSX / CSV	Test cases, final results, metrics, and traceability
Google Chrome	Primary browser for UI execution
Microsoft Edge	Basic cross-browser smoke evidence
Selenium WebDriver + .NET 8 + xUnit	Basic UI automation
Postman	API request design and execution support
Newman	Automated API collection execution and exportable outputs
GitHub Issues	Defect tracking with severity, priority, and labels
Screenshots / TRX / TXT / XML / MP4	Execution evidence and automation proof

CHAPTER 3. TEST DESIGN & EXECUTION

3.1. Test Scenarios

This chapter presents how the test scenarios and test cases were designed for the Online Sales Management System and how the resulting execution evidence supports the final submission. The design phase was based on real system modules, route behavior, role boundaries, validation rules, and business-risk areas identified through source inspection and local execution.

The scenario set was intentionally broader than the minimum number of test cases required by the course. This allowed the team to split ownership cleanly across members, cover both positive and negative flows, and preserve enough business-rule depth to support defect discovery and traceability.

3.1.1. UI test scenarios

The UI scenario design covers the administrative interface and the public product catalog. Administrative scenarios dominate the scope because the highest-risk workflows of the system exist in secured modules such as authentication, permissions, purchases, invoices, stock, reports, and product import. Public UI scenarios were included to verify search, sorting, filtering, and product-detail behavior.

Based on the current scenario matrix, the project contains 33 Admin UI scenarios and 3 Public UI scenarios. These scenarios were then split into detailed executable UI test cases with non-overlapping ownership.

Table 3.1-1: UI scenario distribution by interface

Interface	Scenario count	Main focus
Admin UI	33	Authentication, permissions, CRUD, purchases, invoices, stock, reports, import
Public UI	3	Search, filtering, sorting, and product details
Total UI scenarios	36	Complete UI scenario coverage

Table 3.1-2: Representative UI scenario groups

Module	Representative scenario focus	Risk level
Authentication	Valid login, invalid login, inactive-account blocking	Critical
Permissions	Role-based denial and module visibility	Critical
Products / Import	Validation, duplicate handling, preview and confirm behavior	Critical
Purchases	Draft creation, validation, receiving, cancellation rules	Critical
Invoices	Creation, payment transition, cancellation, stock-sensitive logic	Critical
Stock / Reports	Visibility, filtering, export-source verification	High
Public Catalog	Search, filter, sort, and product details	Medium

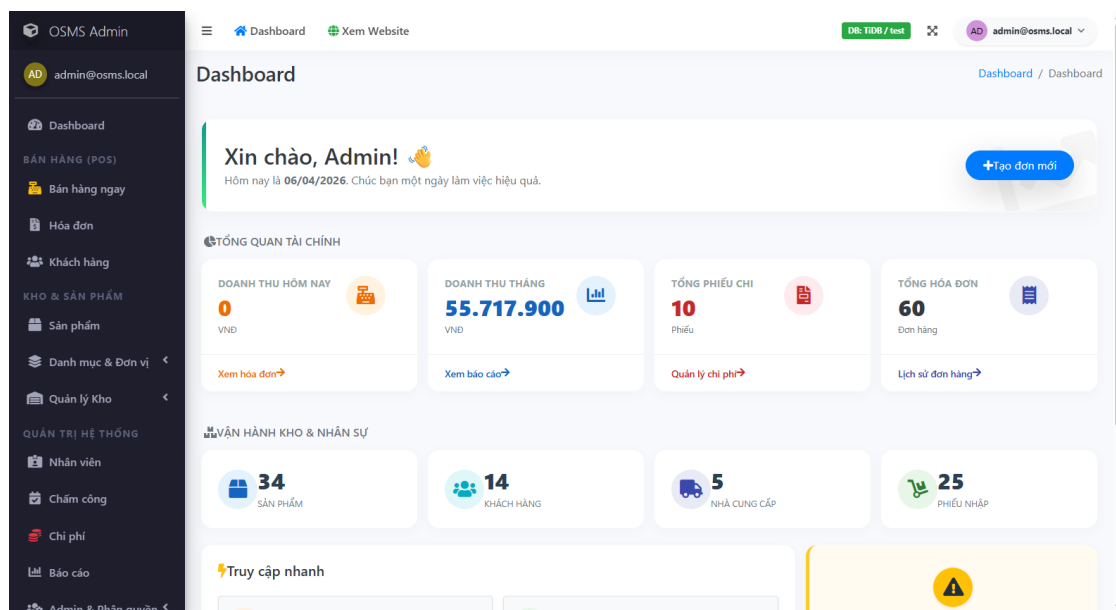


Figure 3.1-1: Example UI execution evidence for a successful authentication scenario.

3.1.2. API test scenarios

API scenario design focused on the real endpoints exposed by the project rather than hypothetical or undocumented services. The scenario set covers the Health API and the public Catalog API. API scenarios were selected to verify service availability, response structure, query handling, detail retrieval, and invalid-parameter behavior.

The current scenario baseline contains 6 API scenarios, which were expanded into 19 detailed API test cases in the dedicated workbook. This split allowed the team to test more than one parameter combination under a single high-level scenario when needed.

Table 3.1-3: API scenario coverage

API scenario area	Scenario focus	Representative endpoint
Health API	Service availability and payload structure	GET /api/v1/health
Catalog list	Pagination, search, filter, and sort behavior	GET /api/v1/catalog/products
Catalog detail	Valid ID and not-found behavior	GET /api/v1/catalog/products/{id}
Catalog metadata	Trending and filter-data retrieval	GET /api/v1/catalog/trending and /filters
Validation handling	Unsupported or invalid query combinations	Catalog endpoints with invalid parameters

```

Newman Full Run Summary
newman

OSMS API Automation

□ 01 Health Smoke
  ↳ TC-API-HLT-001 Health endpoint returns service availability metadata
    GET http://localhost:5068/api/v1/health [200 OK, 318B, 38ms]
    ✓ Status code is 200
    ✓ Health payload contains expected fields

□ 02 Catalog Happy Path
  ↳ TC-API-CAT-001 Catalog products endpoint returns paginated active products with default parameters
    GET http://localhost:5068/api/v1/catalog/products?page=1&pageSize=2 [200 OK, 731B, 318ms]
    ✓ Status code is 200
    ✓ Paginated payload is returned

  ↳ TC-API-CAT-002 Catalog products endpoint supports search by SKU or product name
    GET http://localhost:5068/api/v1/catalog/products?search=SP001&page=1&pageSize=5 [200 OK, 494B, 265ms]
    ✓ Status code is 200
    ✓ Search returns matching products only

  ↳ TC-API-CAT-003 Catalog products endpoint filters by valid categoryId
    GET http://localhost:5068/api/v1/catalog/products?categoryId=30012&page=1&pageSize=5 [200 OK, 949B, 257ms]
    ✓ Status code is 200
    ✓ Every returned item matches the requested category when items are present

  ↳ TC-API-CAT-004 Catalog products endpoint filters by valid brandId
    GET http://localhost:5068/api/v1/catalog/products?brandId=30039&page=1&pageSize=5 [200 OK, 967B, 256ms]

```

Figure 3.1-2: API scenario execution evidence produced by the Newman full-run output.

3.1.3. Edge cases and exception scenarios

Edge cases and exception scenarios were intentionally included to prevent the design from being limited to happy-path verification. These scenarios are important because the rubric rewards logical depth, error handling, and the ability to identify hidden business-rule defects.

- Inactive-account login despite otherwise valid credentials
- Role-based access attempts to secured modules
- Duplicate sku and invalid numeric values in product management
- Non-xlsx and malformed uploads for product import
- Expired or missing preview state during import confirmation
- Purchase creation with missing supplier or empty item list
- Invoice creation with insufficient stock or invalid posted state
- Invoice cancellation and payment transitions under sensitive business states
- Unsupported sort and invalid query combinations in api requests

Table 3.1-4: Representative edge and exception scenarios

Scenario ID	Area	Exception / edge focus	Why it matters
SCN-AUTH-003	Authentication	Inactive account login attempt	Prevents unauthorized account reuse
SCN-GOV-001	Permissions	Sales role opens restricted module	Verifies authorization boundaries
SCN-PROD-006	Product Import	Invalid upload type or malformed workbook	Protects the system from invalid bulk data input
SCN-PROD-007	Product Import	Preview-confirm lifecycle issues	Targets high-risk cache and state behavior
SCN-PUR-002	Purchases	Missing supplier or missing item list	Protects transaction integrity
SCN-INV-003	Invoices	Server-side price and transaction safety	Targets hidden business-rule and security risk
SCN-INV-006	Invoices	Cancellation and stock-return logic	Directly affects inventory consistency
SCN-API-003	Catalog API	Invalid sort / page / page-size validation	Protects API robustness and predictable error handling

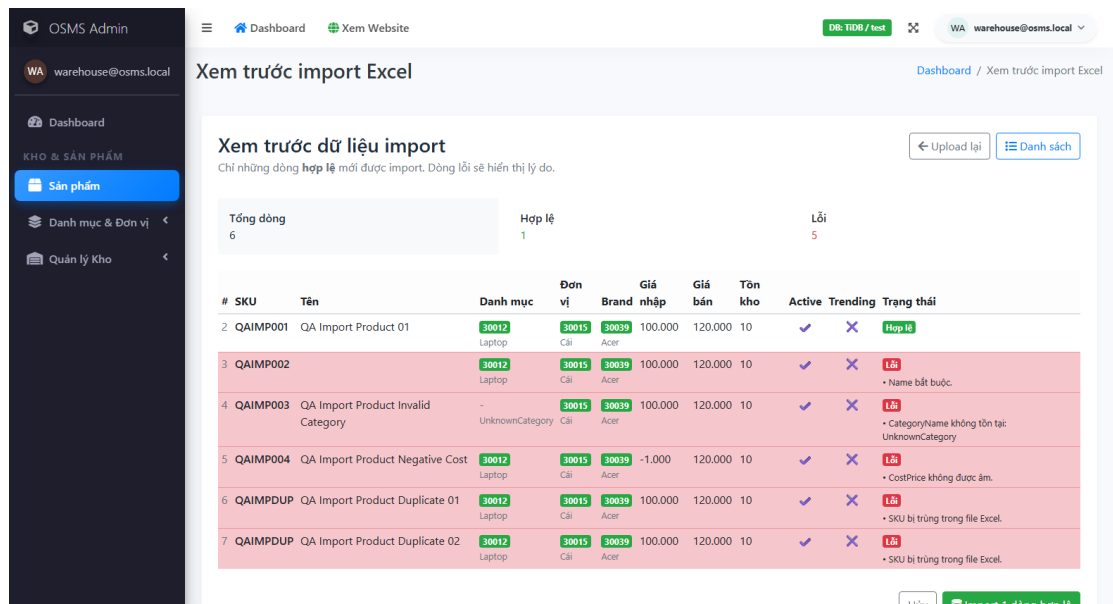


Figure 3.1-3: Example edge-case evidence from the product import preview validation flow.

3.2. Test Case Specification

After the scenario set was stabilized, each scenario was expanded into detailed executable test cases. The final design uses separate UI and API workbooks, with consistent fields for identifiers, preconditions, test data, steps, expected results, actual results, ownership, and evidence references. This structure supports both execution discipline and post-execution traceability.

3.2.1. UI test cases

The UI workbook currently contains 44 detailed UI test cases. These cases cover administrative and public-user flows, and every row includes a direct owner assignment, explicit preconditions, concise steps, expected result, actual result, final status, and evidence reference. The workbook therefore satisfies the course requirement for industry-standard testcase formatting.

Table 3.2-1: Sample UI test case specification (positive case)

Field	Value
Test Case ID	TC-UI-AUTH-001
Scenario ID	SCN-AUTH-001
Title	Login succeeds with valid admin account

Field	Value
Module	Authentication
Owner	Hoang Van Thien
Preconditions	Application is running locally and seeded admin account exists.
Test Data	Email=admin@osms.local; Password=Admin@12345
Steps	1 Open /Admin/Auth/Login; 2 Enter admin email; 3 Enter valid password; 4 Click Sign in.
Expected Result	User is redirected to the admin area dashboard; authenticated session is established; no validation error is shown.
Actual Result	Valid admin login reached the admin dashboard successfully in both the focused Chrome rerun and a secondary Edge smoke rerun on 2026-04-11.
Status	Pass
Evidence / Note	TestResults/Evidence/UI/automation/20260406_053930_TC-UI-AUTH-001-success.png ; TestResults/Evidence/UI/automation/20260411_143741_TC-UI-AUTH-001-success.png ; TestResults/RunnerOutput/UI/auth-permission-rerun.trx ; TestResults/RunnerOutput/UI/edge-auth-smoke.trx ; Videos/OSMS-Automation-Demo.mp4

Table 3.2-2: Sample UI test case specification (defect-focused case)

Field	Value
Test Case ID	TC-UI-INV-001
Scenario ID	SCN-INV-001
Title	Create invoice successfully for walk-in sale with unpaid status
Module	Invoices
Owner	Hoang Van Thien

Field	Value
Preconditions	Sales or admin user is logged in; at least one active product has stock greater than 0.
Test Data	Customer=None; Product=Any active in-stock product; Qty=1; PaidAmount=0
Steps	1 Log in as sales or admin; 2 Open /Admin/Invoices/Create; 3 Leave customer empty for walk-in sale or choose a valid customer; 4 Select one active in-stock product; 5 Set quantity to 1; 6 Set paid amount to 0; 7 Submit the invoice.
Expected Result	Invoice is created successfully; details page is shown; invoice status is Unpaid; stock is reduced by the sold quantity.
Actual Result	Submitting a valid walk-in invoice returned to the Create page with the toast message Failed to create invoice. Please check data and try again. Server log confirms InvalidOperationException caused by a user-initiated transaction with MySqlRetryingExecutionStrategy in InvoicesController.Create line 145. A focused rerun on 2026-04-11 still failed before reaching the details page, so the defect remains reproducible.
Status	Fail
Evidence / Note	TestResults/Evidence/UI/automation/20260406_053902_TC-UI-INV-001-failure.png ; TestResults/Evidence/UI/automation/20260411_143458_TC-UI-INV-001-failure.png ; TestResults/RunnerOutput/UI/invoice-rerun.trx ; TestResults/RunnerOutput/UI/postfix-retest-inv-001.trx ; TestResults/Defects/OSMS-Defect-Log.xlsx ;

Field	Value
	https://github.com/Alucard30Dec/Online-Sales-Management-System/issues/1

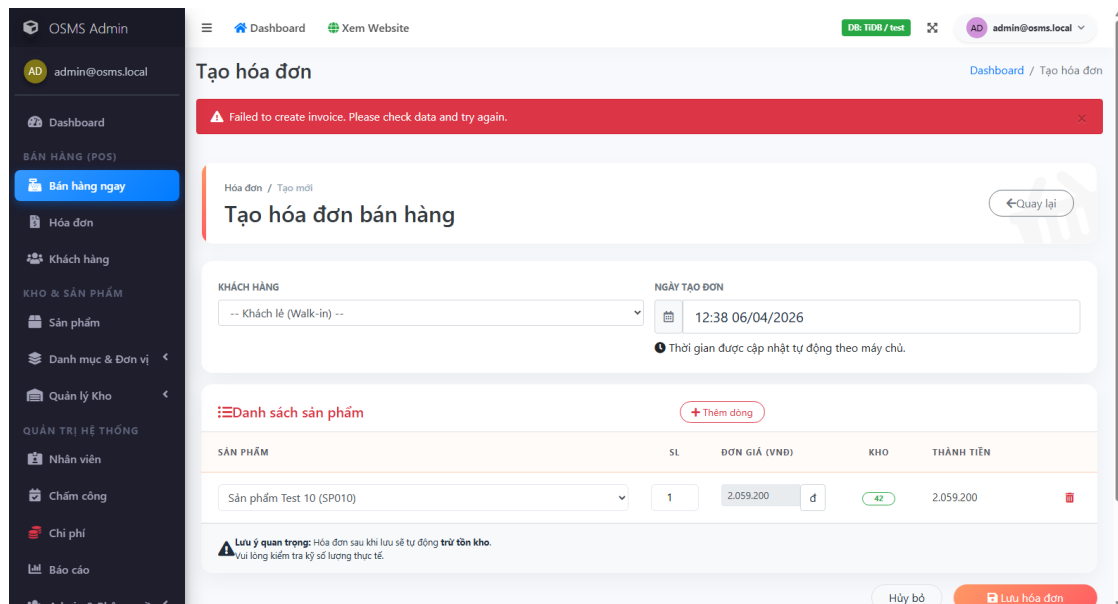


Figure 3.2-1: Defect-focused UI execution evidence for invoice creation failure.

3.2.2. API test cases

The API workbook contains 19 API test cases derived from 6 high-level API scenarios. The API cases use the same traceability principle as the UI cases, but add endpoint-specific fields such as method, authentication requirement, expected status code, and expected response body.

Table 3.2-3: Sample API test case specification

Field	Value
Test Case ID	TC-API-HLT-001
Scenario ID	SCN-API-001
Title	Health endpoint returns service availability metadata
Module	Health API
Endpoint	/api/v1/health
Method	GET
Auth Requirement	None
Test Data	Base URL=http://localhost:5068

Field	Value
Steps	1 Ensure the application is running locally; 2 Send GET request to http://localhost:5068/api/v1/health using Postman or browser.
Expected Status Code	200
Expected Body	JSON response contains non-empty fields for status, service, serverTimeUtc, and version; status value is ok.
Actual Result	Health endpoint returned 200 OK and all Newman assertions for status, service, serverTimeUtc, and version passed in the full collection rerun.
Status	Pass
Evidence / Note	Primary evidence: TestResults/Evidence/API/newman-full-run.txt ; Secondary artifact: TestResults/RunnerOutput/API/newman-results.xml

3.2.3. Test case allocation by member

Ownership allocation was used to satisfy the requirement that each member contributes at least ten non-overlapping test cases. UI cases were distributed by module cluster, while API cases were executed as a shared QA-team artifact due to their smaller surface and automation-centric nature.

Table 3.2-4: Test case allocation by member

Owner	Assigned UI cases	Allocation note
Hoang Van Thien	17	Authentication, permissions, admin, invoices, reports, and partial public-catalog coverage
Nguyen Thanh Dat	13	Customers, suppliers, purchases, and partial stock coverage
Le Quang Duy	14	Products, product import, public catalog, and partial stock coverage
QA Team	19	Shared ownership for API execution and automation evidence

This allocation keeps module ownership visible while preserving a shared quality baseline across the final package. It also supports defense readiness because each member can explain both the rationale and the execution evidence of the areas they owned.

3.2.4. Execution evidence summary

The final execution set is fully recorded in the final-results workbook and its linked evidence files. Every designed test case in the current report has a recorded execution outcome, and each failing case can be traced to supporting screenshots, runner outputs, and defect records. This makes the execution section easier to verify during review and presentation.

Table 3.2-5: Execution evidence summary

Metric	Value
Total executed test cases	63
Pass	56
Fail	7
Admin UI cases	41
Public UI cases	3
API cases	19
Execution completeness	100% of the current baseline

Table 3.2-6: Execution evidence types

Evidence type	Purpose	Representative artifact
UI screenshots	Visual proof of page state and result outcome	Admin login, import preview, invoice failure screenshots
TRX runner output	Execution trace for focused UI automation reruns	UI rerun files under TestResults/RunnerOutput/UI
Newman TXT and XML	Exportable API automation proof	newman-full-run.txt and newman-results.xml
GitHub Issues	Professional bug-management evidence	Live issues for the four confirmed defects

Evidence type	Purpose	Representative artifact
Automation video	Presentation-ready execution walkthrough	OSMS-Automation-Demo.mp4

CHAPTER 4. DEFECT REPORT & METRICS

4.1. Bug Management Tool and Workflow

This chapter explains how defects were controlled, documented, and summarized during the testing process. Because the course requires a real bug-management tool, the project used GitHub Issues as the official issue-tracking platform for confirmed defects. Defect records were synchronized between the issue tracker, the defect register, the final results workbook, and the supporting screenshots and runner outputs.

4.1.1. Selected tool

GitHub Issues was selected as the official bug-management tool for this submission. The tool is appropriate for the project because it supports issue IDs, labels, screenshots, comments, status tracking, and direct linking from the final result package. It also satisfies the course requirement to use a real issue-management platform rather than a manually typed bug list only.

The selected tool was used for the four confirmed defects that remained reproducible in the current execution baseline.

Table 4.1-1: Bug-management tool selection

Item	Value
Selected tool	GitHub Issues
Repository	Alucard30Dec/Online-Sales-Management-System
Used for	Confirmed product defects with reproducible evidence
Linked artifacts	Defect register, final results, screenshots, TRX/TXT/XML evidence
Reason selected	Real issue IDs, practical traceability, and direct repository integration

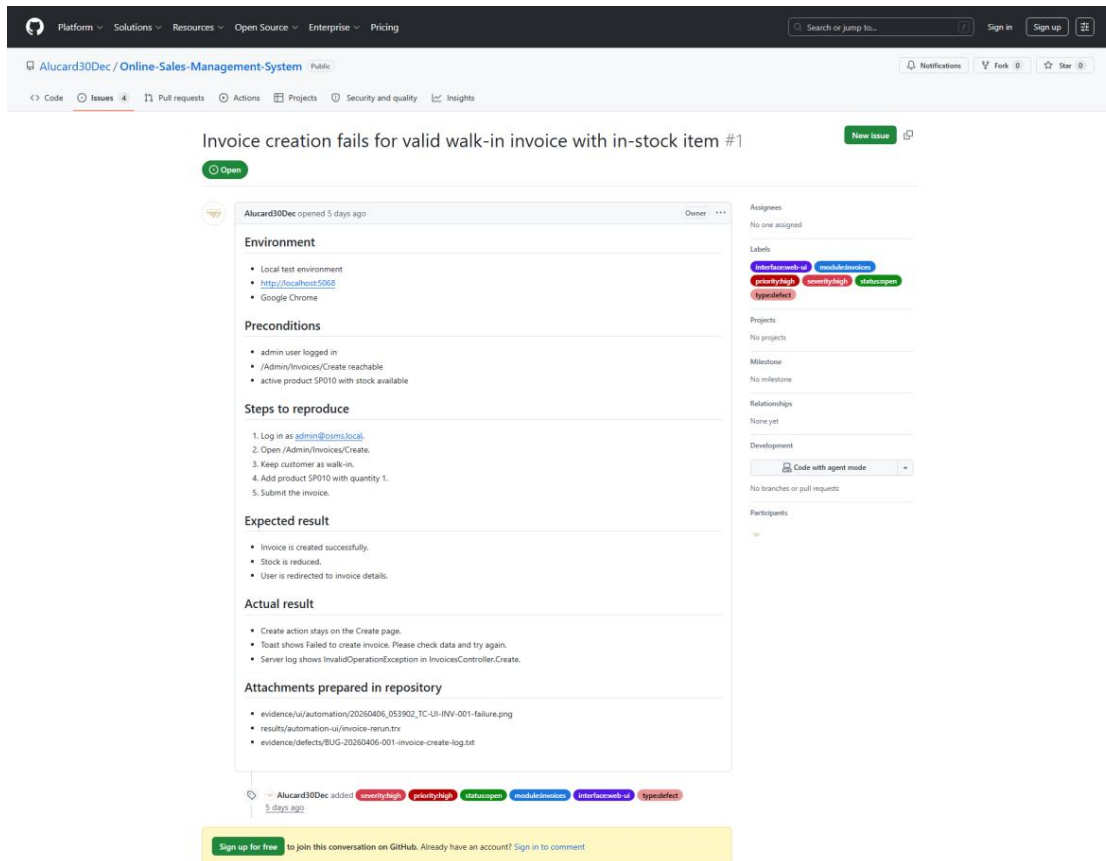


Figure 4.1-1: GitHub Issue example used as the official bug-management record for a confirmed defect.

4.1.2. Issue workflow

The issue workflow used in this submission was designed to keep defect reporting disciplined and evidence-based. An issue was created only after the mismatch was reproducible and supported by execution artifacts.

- Execute or rerun the related test case
- Compare actual behavior against the expected result
- Collect screenshots and runner outputs
- Classify the result as observation or confirmed defect
- Open a github issue for reproducible confirmed defects only
- Link the issue back to the defect register and final results workbook
- Keep the issue open until a real post-fix retest can verify closure

Table 4.1-2: Issue workflow

Step	Action	Expected output
1	Execute or rerun the test case	Observable system result
2	Collect screenshot and runner evidence	Supporting proof files
3	Compare expected and actual behavior	Pass / Fail decision
4	Classify as observation or confirmed defect	Defect triage decision
5	Create GitHub Issue for confirmed defects	Live issue record with labels
6	Sync register, results, and evidence mapping	Full traceability across artifacts
7	Retest after fix when available	Potential closure or continued open status

4.1.3. Severity and priority rules

Severity and priority were assigned according to business impact, reproducibility, and operational risk. The project used simple but defensible rules so that each rating could be justified during presentation defense.

Table 4.1-3: Severity and priority rules

Rule area	How it was applied in this project
Severity: High	Used when the defect blocks a core business workflow such as invoice creation, invoice cancellation, or valid product import confirmation
Severity: Medium	Used when the system blocks invalid input correctly but still presents broken or unreadable validation feedback
Priority: High	Used when the defect affects transaction correctness, inventory, or core operational workflow
Priority: Medium	Used when the defect should be fixed but does not fully break the core transaction path

The actual issue labels used in GitHub include:

- Severity:high / severity:medium
- Priority:high / priority:medium
- Status:open
- Module:invoices / module:purchases / module:product-import
- Interface:web-ui
- Type:defect

4.2. Defect Log

At the final reporting point, the defect register contains 4 confirmed defects and 2 closed observations. The observations are retained for historical traceability, but only the confirmed defects are counted as active product defects in the summary metrics.

4.2.1. Critical and high severity defects

No active critical-severity defect exists in the current result set. However, three high-severity defects remain open and affect core business workflows. These findings are the most important defects in the report because they affect invoice processing, valid import completion, or other stock-sensitive behavior that directly influences business correctness.

Table 4.2-1: High-severity confirmed defects

Defect ID	Related test case(s)	Module	Severity	Priority	Current status	GitHub Issue
BUG-20260406-001	TC-UI-INV-001; TC-UI-INV-003; TC-UI-INV-006	Invoices	High	High	Open - Confirmed	https://github.com/Alucard30Dec/Online-Sales-Management-System/issues/1
BUG-20260411-003	TC-UI-IMP-003	Product Import	High	High	Open - Confirmed	https://github.com/Alucard30Dec/Online-

Defect ID	Related test case(s)	Module	Severity	Priority	Current status	GitHub Issue
						Sales-Management-System/issues/3
BUG-20260411-004	TC-UI-INV-005	Invoices	High	High	Open - Confirmed	https://github.com/Alucard30Dec/Online-Sales-Management-System/issues/4

OSMS Admin

admin@osms.local

Dashboard / Xem Website

DB: TID6 / test

admin@osms.local

Dashboard / Tạo hóa đơn

Failed to create invoice. Please check data and try again.

Hóa đơn / Tạo mới

Tạo hóa đơn bán hàng

← Quay lại

KHÁCH HÀNG: -- Khách lẻ (Walk-in) --

NGÀY TẠO ĐƠN: 12:38 06/04/2026

Thời gian được cập nhật tự động theo máy chủ.

Danh sách sản phẩm

Thêm dòng

SẢN PHẨM	SL	ĐƠN GIÁ (VNĐ)	KHO	THÀNH TIỀN
Sản phẩm Test 10 (SP010)	1	2.059.200 đ	42	2.059.200

Lưu ý quan trọng: Hóa đơn sau khi lưu sẽ tự động trừ tiền kho. Vui lòng kiểm tra kỹ số lượng thực tế.

Hủy bỏ Lưu hóa đơn

Figure 4.2-1: Reproduction evidence for the high-severity invoice creation defect.

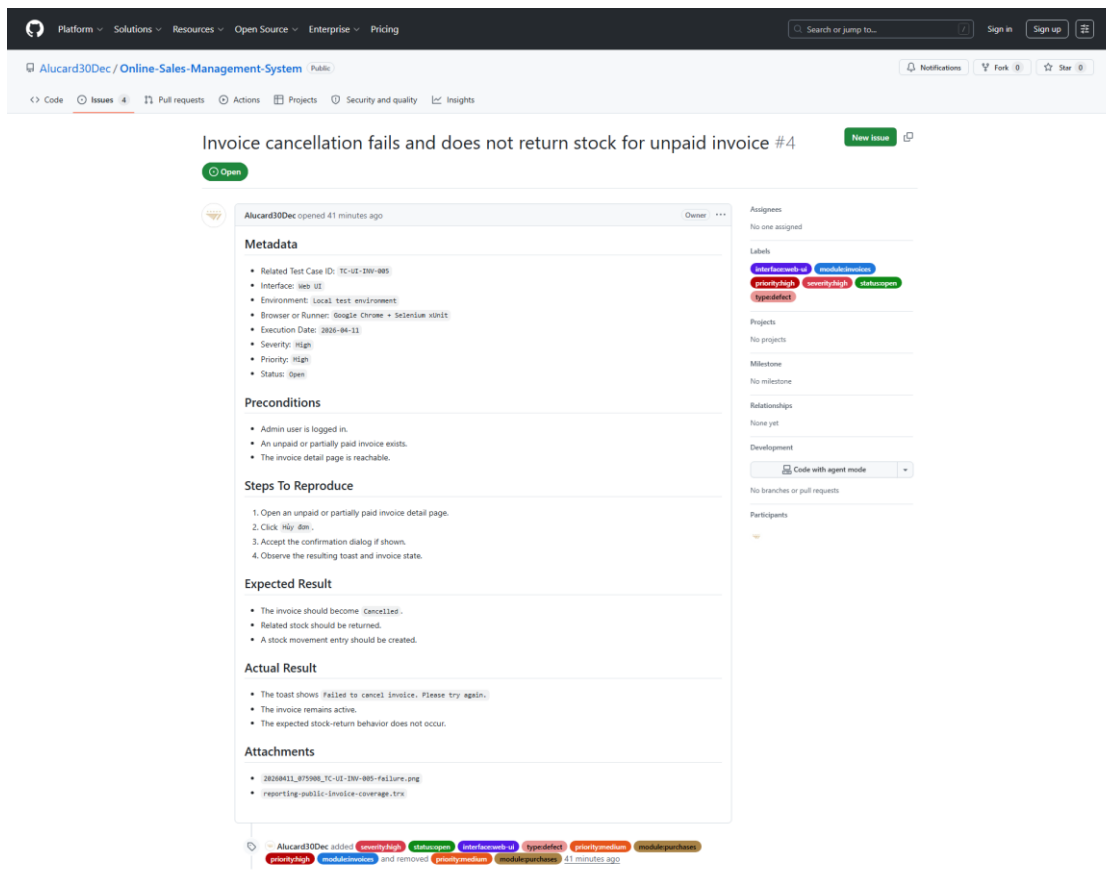


Figure 4.2-2: GitHub Issue evidence for a high-severity invoice cancellation defect.

4.2.2. Medium and low severity defects

The current result set contains one medium-severity confirmed defect and no low-severity confirmed defect. Although this defect does not break the blocking logic of the purchase-create flow, it still reduces usability by showing an unreadable validation banner instead of clear feedback to the user.

Table 4.2-2: Medium and low severity confirmed defects

Defect ID	Related test case(s)	Module	Severity	Priority	Current status	GitHub Issue
BUG-20260411-002	TC-UI-PUR-002; TC-UI-	Purchases	Medium	Medium	Open - Confirmed	https://github.com/Alucard30Dec/Online-Sales-Management-System/issues/2

Defect ID	Related test case(s)	Module	Severity	Priority	Current status	GitHub Issue
	PUR-003					
Low severity defects	None	-	Low	-	-	-

Purchase create validation banner renders without readable text for missing supplier or items #2

Alucard30Dec opened 42 minutes ago

Metadata

- Related Test Case ID: TC-UI-PUR-002, TC-UI-PUR-003
- Interface: ias UI
- Environment: Local test environment
- Browser or Runner: Google Chrome, Selenium v4.0.0
- Execution Date: 2020-04-18
- Severity: Medium
- Priority: Medium
- Status: Open

Preconditions

- Admin user is logged in.
- Purchase create page is reachable.
- Seeded supplier and product data are available.

Steps To Reproduce

1. Open `/kalex/purchases/create`.
2. Submit the form with product lines but without selecting a supplier.
3. Observe the red validation banner on the create page.
4. Repeat the test by selecting a supplier but leaving the purchase without valid items.
5. Observe the validation banner again.

Expected Result

- The create form should remain on the page.
- The validation banner should display readable validation text explaining the missing supplier or missing item requirement.

Actual Result

- The form is blocked as expected.
- However, the red validation banner renders without readable validation text for both the missing-supplier and missing-item scenarios.

Attachments

- 28208430_133496_TC-UI-PUR-002-failure.png
- 28208430_133214_TC-UI-PUR-003-failure.png
- purchase-product-import-coverage-runui.trx

Figure 4.2-3: GitHub Issue evidence for the medium-severity purchase validation defect.

4.2.3. Screenshots and reproduction evidence

Each confirmed defect in the current report is supported by reproducible steps and at least one screenshot or runner artifact. The stronger defect packages also include focused rerun evidence and direct GitHub Issue screenshots. This makes the defect records easier to review

and helps distinguish real product defects from temporary automation or expectation problems.

Table 4.2-3: Defect evidence mapping summary

Defect ID	Evidence types used	Purpose
BUG-20260406-001	UI screenshots, TRX, server log excerpt, GitHub Issue screenshot	Proves invoice-create failure is reproducible and not limited to one scenario
BUG-20260411-002	UI screenshots, focused rerun TRX, GitHub Issue screenshot	Proves purchase validation banner defect is reproducible
BUG-20260411-003	UI screenshots, focused rerun TRX, GitHub Issue screenshot	Proves import confirmation still fails after valid preview
BUG-20260411-004	UI screenshots, focused rerun TRX, GitHub Issue screenshot	Proves invoice cancellation still fails and stock return does not occur

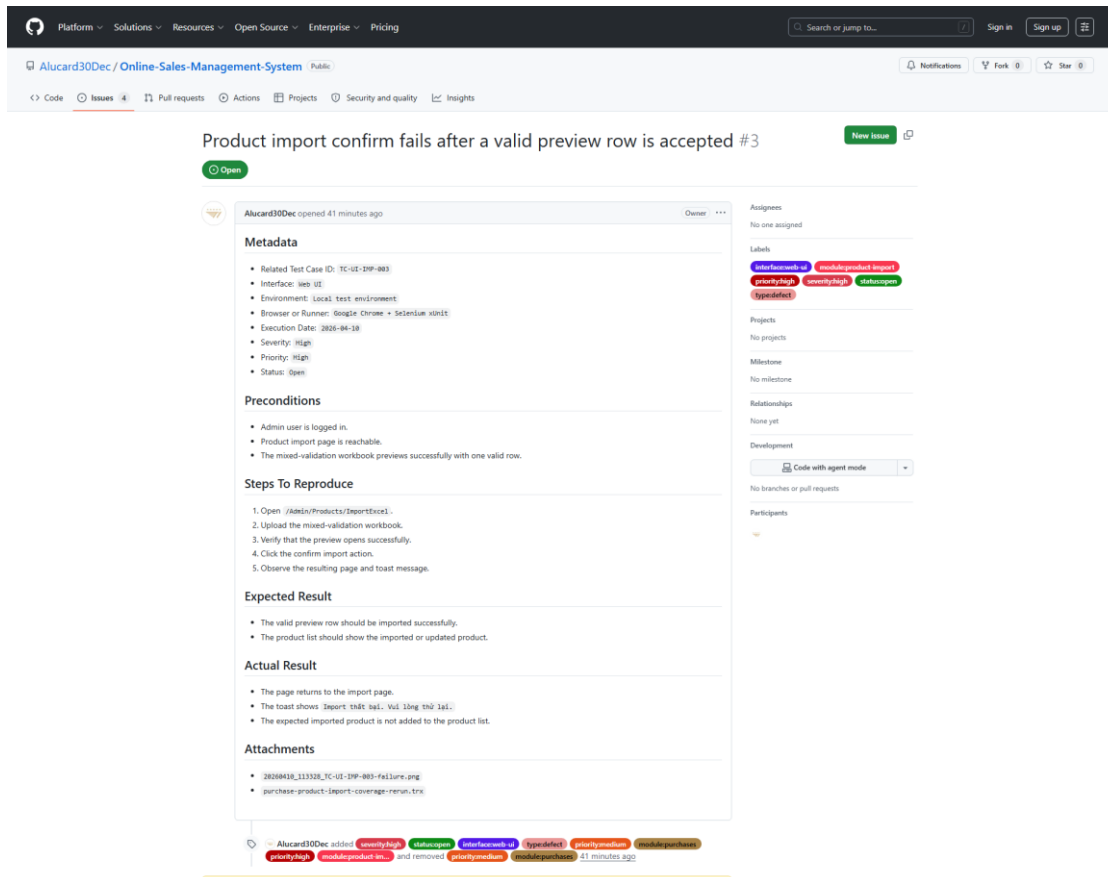


Figure 4.2-4: GitHub Issue evidence for the product-import confirmed defect.

4.3. Test Summary Metrics

The test summary metrics describe the final execution state of the project. It is important to distinguish between failing test cases and unique confirmed defects. In this report, 7 test cases currently fail, but those failures map to 4 confirmed defects because several failing cases are caused by the same underlying issue.

4.3.1. Total test cases executed

The current report contains 63 executed test cases. According to the synchronized final-results workbook, all designed UI and API cases in scope now have recorded execution outcomes, which means execution completeness reached 100% for the documented baseline.

Table 4.3-1: Execution totals

Metric	Value
Total test cases	63
Executed test cases	63

Metric	Value
Execution progress	100%

4.3.2. Pass/Fail/Blocked statistics

The final execution results contain 56 passing test cases, 7 failing test cases, and 0 blocked test cases. This corresponds to a pass rate of 88.89% and a fail rate of 11.11%.

Table 4.3-2: Pass / Fail / Blocked statistics

Status	Count	Interpretation
Pass	56	Expected behavior matched real execution evidence
Fail	7	Observed behavior did not match the expected result
Blocked	0	No blocked cases remain in the current baseline

4.3.3. Defect count by severity

Severity distribution should be interpreted using confirmed defects only, not raw failing test cases. The current confirmed-defect profile shows concentration in high-severity business workflows, especially invoice and product-import related behavior.

Table 4.3-3: Confirmed defect count by severity

Severity	Confirmed defect count
Critical	0
High	3
Medium	1
Low	0

4.3.4. Defect ratio and observations

The confirmed-defect ratio in the current report is 6.35%, calculated as 4 confirmed defects out of 63 executed test cases. This is lower than the fail-case rate of 11.11% because several failing test cases point to the same underlying defect. This distinction is important for accurate reporting because fail counts and defect counts do not always represent the same level of product impact.

In addition to the confirmed defects, the defect register also records 2 observations that were closed after rerun analysis. These records remain useful because they show that the team did not automatically classify every unexpected behavior as a product defect. Instead, each mismatch was reviewed and reclassified based on reproducible evidence.

Table 4.3-4: Defect ratio and observations

Metric	Value	Note
Confirmed defects	4	Unique product defects currently open
Failing test cases	7	Some failures share the same root cause
Defect ratio	6.35%	Confirmed defects / executed test cases
Fail-case rate	11.11%	Failing cases / executed test cases
Closed observations	2	Historical mismatch records resolved by rerun analysis

OSMS Test Metrics Summary

Execution baseline synchronized to Powered by GPT package on 2026-04-11

Total Cases 63	Executed 63	Pass 56	Fail 7	Not Run 0
---------------------------------	------------------------------	--------------------------	-------------------------	----------------------------

Execution quality highlights

Execution Progress: 100%
Scenario Execution Coverage: 100%
Confirmed Defects: 4

Status interpretation

Pass = expected behavior confirmed by runtime evidence.
Fail = runtime evidence showed a mismatch between expected and actual behavior.
Not Run = 0 in the current synchronized baseline.

Figure 4.3-1: Metrics summary exported from the synchronized results workbook

CHAPTER 5. CONCLUSION AND FUTURE WORK

5.1. Achievements and compliance with requirements

The final submission met the main academic and practical goals of the Software Testing project. The team produced a complete testing package that includes the report, presentation slides, scenario and test-case workbooks, final execution results, metrics, defect records, automation assets, supporting screenshots, runner outputs, and an automation demo video. These artifacts were synchronized so that the main claims in the report can be traced back to real execution outputs.

From an execution perspective, the project reached full recorded coverage with 63/63 executed test cases, 56 passing cases, 7 failing cases, 0 blocked cases, and no cases left as Not Run. Scenario coverage also reached 42/42, which means every documented scenario in scope was mapped and executed in the final evidence set.

The team also met the mandatory course requirements regarding individual contribution and defect handling. UI test-case ownership remained non-overlapping across members, each member contributed more than ten UI test cases, and confirmed defects were managed through a real GitHub Issues workflow with severity, priority, screenshots, and reproducible steps.

Table 5.1-1: Final requirement compliance summary

Requirement area	Current status	Supporting evidence
Group-based final testing project	Satisfied	Team allocation tables and shared submission package
At least 10 non-overlapping test cases per member	Satisfied	UI ownership counts: Hoang Van Thien=17, Nguyen Thanh Dat=13, Le Quang Duy=14
Industry-style test case fields	Satisfied	UI and API Excel/CSV test-case files
Execution evidence	Satisfied	Screenshots, TRX, TXT, XML, and final result workbook
Real bug-management tool	Satisfied	4 live GitHub Issues linked from the defect register

Requirement area	Current status	Supporting evidence
Automation bonus support	Satisfied	Selenium UI runs, Newman outputs, and MP4 demo video
Submission packaging	Satisfied	MainReport, README.md, TestCases, TestResults, TestScript-Data, Videos

Table 5.1-2: Final delivery baseline

Measure	Value	Interpretation
Total test cases	63	Combined UI and API baseline
Executed test cases	63	Execution progress 100%
Pass / Fail / Blocked	56 / 7 / 0	Pass rate 88.89%, fail rate 11.11%
Scenario mapping	42/42	All documented scenarios mapped to test cases
Executed scenarios	42/42	Scenario execution coverage 100%
Confirmed defects	4	Backed by GitHub Issues and screenshots

OSMS Test Metrics Summary

Execution baseline synchronized to Powered by GPT package on 2026-04-11

Total Cases 63	Executed 63	Pass 56	Fail 7	Not Run 0
---------------------------------	------------------------------	--------------------------	-------------------------	----------------------------

Execution quality highlights

Execution Progress: 100%
Scenario Execution Coverage: 100%
Confirmed Defects: 4

Status interpretation

Pass = expected behavior confirmed by runtime evidence.
Fail = runtime evidence showed a mismatch between expected and actual behavior.
Not Run = 0 in the current synchronized baseline.

Figure 5.1-1: Final execution and metrics summary used to support the conclusion of the testing project.

5.2. Challenges and limitations

Although the submission achieved full execution coverage for the documented test-case baseline, several limitations remain. These limitations do not invalidate the test results, but they explain why the current package should be described as a strong and evidence-backed submission rather than as proof that the product is defect-free.

- 4 confirmed defects remain open at the reporting point, including 3 high-severity defects and 1 medium-severity defect.
- The project captured defect discovery and live issue tracking, but post-fix retest evidence is still unavailable because the underlying product defects were not fixed within the current cycle.
- Cross-browser evidence exists only at a basic smoke level on Microsoft Edge; it is not a full compatibility certification matrix.
- The system was tested in a controlled local environment rather than a production-like staging environment.
- Non-functional areas such as load, penetration, accessibility, and recovery testing were intentionally excluded from scope.

The most important practical limitation is that the defect lifecycle is not fully closed. The team successfully identified, documented, and classified defects, but closure-level verification still depends on future product fixes followed by a controlled retest cycle.

Table 5.2-1: Current limitations and impact

Limitation	Current impact	Reason it remains open
Open confirmed defects	Product cannot be claimed defect-free	Fixes were not completed before the final reporting point
No post-fix pass retest	Defect lifecycle is not fully closed	No new build with verified fixes was available
Limited browser matrix	Compatibility claims must remain conservative	Only Chrome baseline plus Edge smoke were executed
Local environment only	Deployment-specific behavior was not certified	The course project was executed in a local environment
Out-of-scope non-functional testing	Performance and security conclusions are limited	These areas were outside the official functional scope

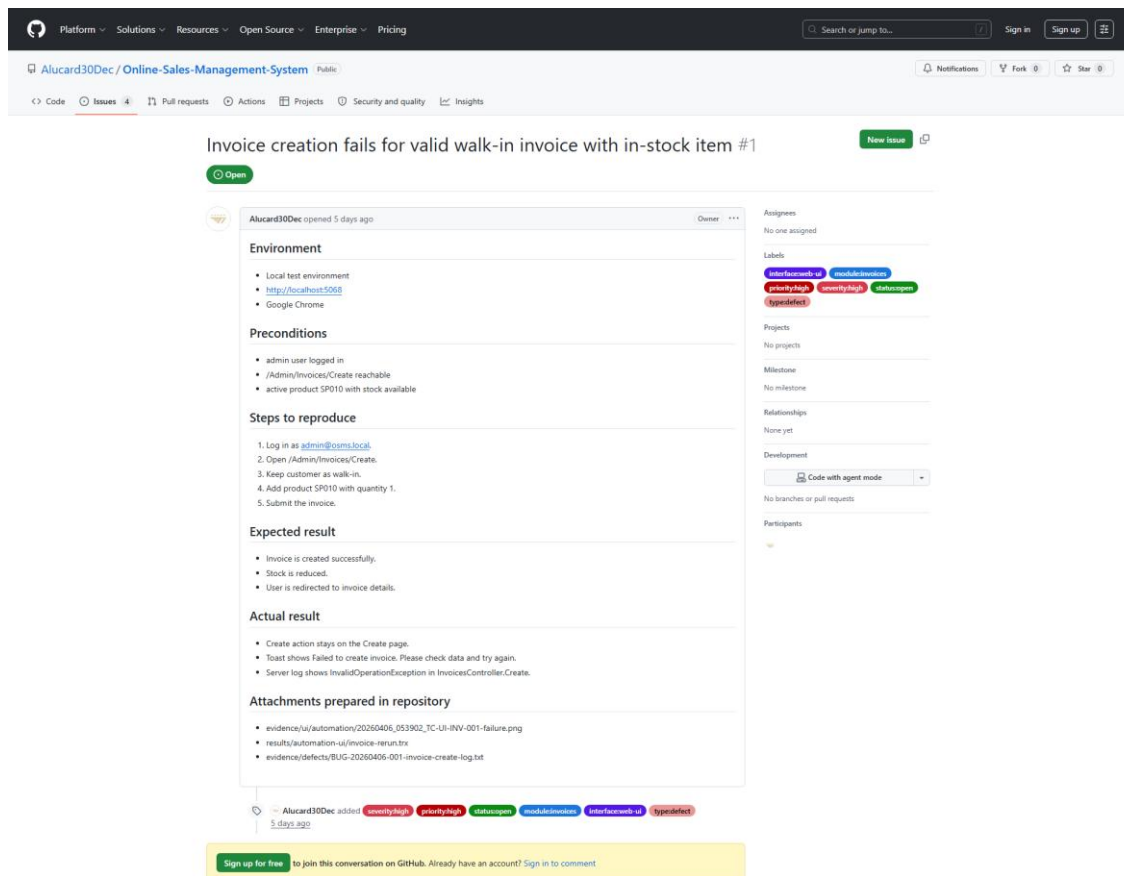


Figure 5.2-1: Example of a confirmed open defect that remains unresolved at the final reporting point.

5.3. Suggestions for future enhancements

Future work should focus first on product risk reduction and then on process maturity. The next cycle should prioritize defect fixing and closure, because this is the shortest path to improving actual software quality rather than only improving documentation quality.

- Fix the four confirmed open defects and perform controlled post-fix retests with updated evidence.
- Expand browser coverage beyond Chrome and the current Edge smoke run to build a stronger compatibility baseline.
- Add more negative and resilience-oriented API tests, especially for invalid filters, unsupported values, and boundary conditions.
- Broaden automation coverage for high-value regression flows while keeping outputs mapped to the official test-case IDs.

- Introduce CI-based automation execution so that UI and API smoke runs can be reproduced more consistently after future changes.

From an academic point of view, the submission already demonstrates structured planning, evidence-based execution, real defect management, and practical automation support. From an engineering point of view, the next cycle should focus on closing the defect lifecycle by fixing the open defects and validating the fixes through controlled retests.

Table 5.3-1: Recommended future enhancement roadmap

Priority	Enhancement	Expected benefit
1	Fix open confirmed defects and rerun retests	Closes the defect lifecycle and improves product stability
2	Extend browser compatibility verification	Improves confidence in UI consistency across browsers
3	Increase API negative-path automation	Strengthens backend validation coverage
4	Expand UI regression automation	Improves repeatability for critical business flows
5	Move automation to CI or scheduled reruns	Improves repeatable verification after changes

Newman Full Run Summary

newman

OSMS API Automation

□ 01 Health Smoke

└ TC-API-HLT-001 Health endpoint returns service availability metadata

GET http://localhost:5068/api/v1/health [200 OK, 318B, 38ms]

✓ Status code is 200

✓ Health payload contains expected fields

□ 02 Catalog Happy Path

└ TC-API-CAT-001 Catalog products endpoint returns paginated active products with default parameters

GET http://localhost:5068/api/v1/catalog/products?page=1&pageSize=2 [200 OK, 731B, 318ms]

✓ Status code is 200

✓ Paginated payload is returned

└ TC-API-CAT-002 Catalog products endpoint supports search by SKU or product name

GET http://localhost:5068/api/v1/catalog/products?search=SP001&page=1&pageSize=5 [200 OK, 494B, 265ms]

✓ Status code is 200

✓ Search returns matching products only

└ TC-API-CAT-003 Catalog products endpoint filters by valid categoryId

GET http://localhost:5068/api/v1/catalog/products?categoryId=30012&page=1&pageSize=5 [200 OK, 949B, 257ms]

✓ Status code is 200

✓ Every returned item matches the requested category when items are present

└ TC-API-CAT-004 Catalog products endpoint filters by valid brandId

GET http://localhost:5068/api/v1/catalog/products?brandId=30039&page=1&pageSize=5 [200 OK, 967B, 256ms]

Figure 5.3-1: Existing automation evidence that can be expanded in future iterations to increase regression strength.

Overall, the final submission meets the required deliverables and reflects a credible software-testing process supported by real execution outputs, defect records, metrics, and GitHub-based issue tracking. The package is ready for submission, and the team can explain the reported results clearly during the presentation. The remaining work is mainly product-side: fixing the confirmed defects and rerunning focused regression checks after those fixes are available.

APPENDIX

Appendix 1. GitHub

The final submission package is stored in the project repository under the clean submission folder so that the examiner can access the report, test cases, results, automation assets, and video from one consistent location.

Table A.1: GitHub submission links

Item	Link / location
GitHub Repository	https://github.com/Alucard30Dec/Online-Sales-Management-System
Clean submission package	https://github.com/Alucard30Dec/Online-Sales-Management-System/tree/main/Report%20Test%20subject/Powered%20by%20GPT
Top-level folders in clean package	MainReport, README.md, TestCases, TestResults, TestScript-Data, Videos

Appendix 2. Excel test cases

The official test-case artifacts are maintained in Excel-compatible workbook and CSV form. These files are used as the main reference for scenario mapping, ownership, execution details, and cross-checking with the final result package.

Table A.2: Excel and scenario artifacts

Artifact	Package-relative path	Purpose
UI test cases workbook	TestCases/UI/OSMS-UI-Test-Cases.xlsx	44 UI test cases with ownership, expected and actual results, and status
API test cases workbook	TestCases/API/OSMS-API-Test-Cases.xlsx	19 API test cases with request and result coverage
UI test cases CSV	TestCases/UI/OSMS-UI-Test-Cases.csv	CSV export for traceability and automation-friendly review

Artifact	Package-relative path	Purpose
API test cases CSV	TestCases/API/OSMS-API-Test-Cases.csv	CSV export for API case review
Scenario list	TestCases/Scenarios/test-scenarios.md	Documented UI and API scenario set used to derive the test cases

Appendix 3. Test script and test data

The automation scripts and prepared test data used in the final submission are stored in the TestScript-Data folder. These artifacts support manual reruns, UI automation, API automation, and data-driven validation scenarios such as product import.

Table A.3: Test scripts and test data

Artifact group	Package-relative path	Notes
Automation root	TestScript-Data/Automation/	Contains UI and API automation assets
UI automation	TestScript-Data/Automation/ui/	Selenium WebDriver + .NET 8 + xUnit test project
API automation	TestScript-Data/Automation/api/	Postman collection, environment, and Newman run script
Test data root	TestScript-Data/TestData/	Prepared seeded data and import files
Account data	TestScript-Data/TestData/accounts/OSMS-Test-Accounts.md	Seeded accounts used for role-based verification
Import data	TestScript-Data/TestData/ui/	Mixed-validation and negative import files
API data	TestScript-Data/TestData/api/OSMS-API-Test-Data.json	Reference values for API execution

Verified tools used in the submission package:

- Visual Studio / .NET 8 tooling for application build and UI automation execution
- Google Chrome and Microsoft Edge for browser-based execution
- Postman and Newman for API execution
- GitHub Issues for defect management
- Excel-compatible workbooks and CSV files for test cases, final results, and metrics
- Microsoft Word and PowerPoint for the final report and presentation deliverables

Generative AI was used only in a limited supporting role for requirement summarization, naming suggestions, grammar review, and coding guidance. It was not used to fabricate execution evidence, defects, or final results. All submission-facing content was reviewed manually by the team and aligned with real execution artifacts before inclusion in the report.

Appendix 4. Final results workbook and images

The final execution results are documented in the result workbook and the supporting evidence folders. These files are used to support the pass, fail, and defect claims presented in the report.

Table A.4: Final results and evidence package

Artifact	Package-relative path	Purpose
Final results workbook	TestResults/FinalResults/OSMS-Final-Test-Results.xlsx	Records 63 executed cases with final status
Final results CSV	TestResults/FinalResults/OSMS-Final-Results.csv	Summary baseline: 56 Pass / 7 Fail
Execution-evidence mapping	TestResults/FinalResults/execution-evidence-mapping.csv	Links test cases to screenshots, runner outputs, and issues
Metrics workbook	TestResults/Metrics/OSMS-Test-Metrics.xlsx	Aggregated metrics and summary views
UI evidence	TestResults/Evidence/UI/automation/	Screenshots proving UI execution outcomes
API evidence	TestResults/Evidence/API/newman-full-run.txt	Text summary of Newman API execution
Report evidence	TestResults/Evidence/Report/	Screenshots of metrics and report-support visuals

Appendix 5. Automation video

The submission includes a recorded automation video used to demonstrate UI and API execution evidence in a concise presentation-friendly format.

Table A.5: Automation video artifacts

Artifact	Package-relative path	Purpose
Automation demo video	Videos/OSMS-Automation-Demo.mp4	Primary automation demonstration artifact
Video description	Videos/Automation-Video-Description.md	Explains covered flows and evidence mapping
Recording script	Videos/record-automation-demo.ps1	Repeatable capture workflow for the demo package

Appendix 6. Bug tracker screenshots

Bug-management evidence is stored in both workbook and screenshot form. The defect package combines the defect register, the workbook log, and four live GitHub Issues for the confirmed defects included in the final report.

Table A.6: Bug tracker and defect artifacts

Artifact	Package-relative path	Notes
Defect register CSV	TestResults/Defects/OSMS-Defect-Register.csv	4 confirmed defects plus closed observations
Defect log workbook	TestResults/Defects/OSMS-Defect-Log.xlsx	Submission-facing defect tracking workbook
GitHub issue screenshots	TestResults/Evidence/Defects/	Screenshots for issues #1, #2, #3, and #4
GitHub issue checklist files	TestResults/Defects/GitHubIssues/	Structured issue drafts and evidence notes

REFERENCES

Bootstrap. (n.d.). Get started with Bootstrap. Bootstrap v5.3. Retrieved April 12, 2026, from <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

GitHub. (n.d.). About Git. GitHub Docs. Retrieved April 12, 2026, from <https://docs.github.com/en/get-started/using-git/about-git>

Microsoft. (2023, January 12). Migrations overview. Microsoft Learn. <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/>

Microsoft. (2024, June 17). Overview of ASP.NET Core MVC. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/mvc/overview>

Microsoft. (2024, November 12). Overview of Entity Framework Core. Microsoft Learn. <https://learn.microsoft.com/en-us/ef/core/>

Microsoft. (2025, July 30). Overview of ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/overview>

Microsoft. (2025, November 12). Introduction to Identity on ASP.NET Core. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/security/authentication/identity>

TASK DIVISION AND CONTRIBUTION SUMMARY

The following summary reflects the primary UI test-case ownership used in the final report, together with the main integration responsibilities visible in the submission package. The workload percentage is based on non-overlapping UI ownership because that is the clearest and most verifiable individual contribution in the current artifacts.

Table T.1: Task division and contribution summary

Member name	Assigned tasks	Workload contribution (%)	Completion status (%)
Hoàng Văn Thiên	Project audit, traceability alignment, final integration, authentication, permissions, admin management, invoices, reports, partial public catalog	33.33%	100%
Nguyễn Thành Đạt	Customers, suppliers, purchases, partial stock workflows	33.33%	100%
Lê Quang Duy	Products, product import, partial stock, public catalog workflows	33.33%	100%

Note: the workload contribution percentage above is calculated from primary UI ownership only. Shared API execution, report consolidation, and package synchronization were completed collaboratively and are therefore described in the assigned-task column instead of being redistributed numerically.